

ΚΕΦΑΛΑΙΟ 1

Εισαγωγή στη Fortran

Βασική δομή προγράμματος • Μεταβλητές • Τύποι δεδομένων • Εκχώρηση •
Εκφράσεις • Είσοδος/Έξοδος

Στόχος του κεφαλαίου

Ο φοιτητής θα μάθει να διαβάζει ένα απλό πρόγραμμα Fortran, να αναγνωρίζει τα βασικά του μέρη και να μετατρέπει μια απλή μαθηματική ή φυσική εκφώνηση σε πρόγραμμα. Η έμφαση δίνεται στην κατανόηση και όχι στην αποστήθιση.

Βάση του κεφαλαίου

Οι σημειώσεις έχουν οργανωθεί με βάση το υλικό του μαθήματος: FORTRAN_2025, Περιγραφή Μαθήματος, Ασκήσεις 01 και Εργαστήριο 2 — Εκφράσεις και Προτάσεις / Μορφοποίηση. Η διατύπωση έχει προσαρμοστεί ώστε να απευθύνεται απευθείας στον φοιτητή και να λειτουργεί ως προσωπικό εγχειρίδιο μελέτης.

Τρόπος μελέτης

Ο φοιτητής καλό είναι να διαβάζει πρώτα την εξήγηση, να εκτελεί τα παραδείγματα μόνος του και στη συνέχεια να λύνει τις ασκήσεις χωρίς να κοιτάζει αμέσως τη λύση. Σκοπός είναι να μπορεί στο τέλος να αναλύει μόνος του μια εκφώνηση και να αποφασίζει τι πρέπει να γράψει.

Περιεχόμενα

1. Τι είναι ένα πρόγραμμα και τι κάνει ο προγραμματιστής
2. Η βασική δομή ενός προγράμματος Fortran
3. Το αλφάβητο και τα ονόματα της Fortran
4. Μεταβλητές και τύποι δεδομένων
5. Η εντολή implicit none
6. Εκχώρηση τιμής
7. Αριθμητικοί τελεστές και εκφράσεις
8. Η ακέραια και η πραγματική διαίρεση
9. Η συνάρτηση MOD
10. Είσοδος και έξοδος: READ και PRINT
11. Από μαθηματικό τύπο σε Fortran
12. Πώς αναλύεται μια άσκηση
13. Συχνά λάθη
14. Ασκήσεις αυτοελέγχου
15. Checklist πριν προχωρήσει

1. Τι είναι ένα πρόγραμμα και τι κάνει ο προγραμματιστής

Ένα πρόγραμμα είναι ένα σύνολο από σαφείς εντολές που εκτελεί ο υπολογιστής με συγκεκριμένη σειρά. Για να λύσει ένα πρόβλημα, ο φοιτητής δεν ξεκινά γράφοντας αμέσως κώδικα. Πρώτα πρέπει να καταλάβει το πρόβλημα και να το σπάσει σε απλά βήματα.

Η βασική λογική είναι:

Δεδομένα → Υπολογισμοί → Αποτελέσματα

Παράδειγμα: Αν ζητείται το εμβαδόν κύκλου και δίνεται η ακτίνα, ο φοιτητής πρέπει να αναγνωρίσει ότι η ακτίνα είναι δεδομένο, ο τύπος είναι $E = \pi r^2$ και το E είναι το ζητούμενο αποτέλεσμα.

- Να αναγνωρίζει ποια δεδομένα δίνονται.
- Να εντοπίζει ποια μεγέθη πρέπει να υπολογιστούν.
- Να γράφει τους μαθηματικούς τύπους πριν από τον κώδικα.
- Να μετατρέπει στη συνέχεια τους τύπους σε συντακτικά σωστές εκφράσεις Fortran.

2. Η βασική δομή ενός προγράμματος Fortran

Το υλικό του μαθήματος παρουσιάζει το κύριο πρόγραμμα ως οργανωμένη μονάδα με αρχή, δηλώσεις δεδομένων, εκτελέσιμες προτάσεις και τέλος. Για τα πρώτα προγράμματα ο φοιτητής θα χρησιμοποιεί κυρίως την απλή μορφή:

```
program όνομα_προγράμματος

    implicit none

    ! δηλώσεις δεδομένων

    ! εκτελέσιμες προτάσεις

end program όνομα_προγράμματος
```

Η πρόταση program δηλώνει την αρχή του προγράμματος και δίνει το όνομά του. Η end program δηλώνει το τέλος. Ανάμεσα σε αυτά βρίσκονται οι δηλώσεις και οι εντολές που θα εκτελεστούν.

Στο υλικό παρουσιάζεται γενικότερα η δομή του κύριου προγράμματος με αρχικές δηλώσεις, δηλώσεις δεδομένων, εκτελέσιμες προτάσεις και, σε πιο προχωρημένο επίπεδο, εσωτερικές ή εξωτερικές διαδικασίες. Στο παρόν κεφάλαιο χρησιμοποιείται μόνο το μέρος που χρειάζεται για τα πρώτα απλά προγράμματα.

3. Το αλφάβητο και τα ονόματα της Fortran

Σύμφωνα με το υλικό, τα ονόματα που δημιουργεί ο προγραμματιστής μπορούν να χρησιμοποιούν γράμματα, αριθμούς και την κάτω παύλα `_`. Το όνομα μπορεί να έχει έως 63 χαρακτήρες και ο πρώτος χαρακτήρας πρέπει να είναι γράμμα.

Σωστά παραδείγματα:

```
x
age
```

```
temperature
initial_velocity
number_of_students
x1
```

Μη αποδεκτά παραδείγματα:

```
1x
2number
_value
```

Καλό είναι τα ονόματα να είναι περιγραφικά. Για παράδειγμα, η `initial_velocity` μεταφέρει πολύ περισσότερη πληροφορία από μια γενική μεταβλητή `x`.

4. Μεταβλητές και τύποι δεδομένων

Τύπος	Τι αποθηκεύει	Παράδειγμα
INTEGER	Ακέραιες τιμές	5, -12, 100
REAL	Πραγματικές τιμές	3.14, -2.5
COMPLEX	Μιγαδικές τιμές	(2.0, 3.0)
LOGICAL	Λογικές τιμές	.TRUE., .FALSE.
CHARACTER	Χαρακτήρες / κείμενο	'Hello'

Δήλωση μεταβλητών:

```
integer :: a, b, c
real :: x, y
logical :: found
character(len=20) :: name
```

Στα πρώτα κεφάλαια ο φοιτητής θα χρησιμοποιήσει κυρίως INTEGER και REAL. Οι δύο τύποι πρέπει να ξεχωρίζουν από την αρχή, επειδή επηρεάζουν τον τρόπο με τον οποίο εκτελούνται οι αριθμητικές πράξεις.

5. Η εντολή `implicit none`

Η Fortran επιτρέπει ιστορικά την υπονοούμενη δήλωση τύπων. Το υλικό του μαθήματος χαρακτηρίζει αυτή την πρακτική κακή πρακτική. Για να υποχρεώσει τον εαυτό του να δηλώνει ρητά όλους τους τύπους, ο φοιτητής χρησιμοποιεί:

```
implicit none
```

Η πρόταση αυτή θα θεωρείται βασικό μέρος κάθε αυτόνομου προγράμματος που γράφει ο φοιτητής. Στις δηλώσεις θα χρησιμοποιεί επίσης τη σύνταξη ::.

6. Εκχώρηση τιμής

Η βασική μορφή εκχώρησης είναι:

```
μεταβλητή = έκφραση
```

Η δεξιά πλευρά υπολογίζεται και το αποτέλεσμα αποθηκεύεται στη μεταβλητή της αριστερής πλευράς.

```
integer :: a, b, c
```

```
a = 2  
b = 3  
c = a + b
```

Μετά την εκτέλεση, η c έχει την τιμή 5.

Ιδιαίτερη προσοχή χρειάζεται στην εντολή:

```
x = x + 1
```

Δεν πρόκειται για μαθηματική ισότητα. Ο φοιτητής πρέπει να τη διαβάζει ως: «πάρε την παλιά τιμή του x, πρόσθεσε 1 και αποθήκευσε τη νέα τιμή στο x». Αυτή η λογική θα είναι απαραίτητη αργότερα στις επαναλήψεις.

7. Αριθμητικοί τελεστές και εκφράσεις

Το υλικό δίνει τους βασικούς αριθμητικούς τελεστές: + πρόσθεση, - αφαίρεση, * πολλαπλασιασμός, / διαίρεση και ** ύψωση σε δύναμη.

Οι εκφράσεις συνδυάζουν μεταβλητές, σταθερές και τελεστές. Η Fortran ακολουθεί κανόνες προτεραιότητας. Για να κάνει τη σειρά των πράξεων σαφή, ο φοιτητής πρέπει να χρησιμοποιεί παρενθέσεις όπου χρειάζεται.

```
x = 5 + 6 * 7  
y = (5 + 6) * 7
```

Στην πρώτη έκφραση προηγείται ο πολλαπλασιασμός, ενώ στη δεύτερη οι παρενθέσεις επιβάλλουν πρώτα την πρόσθεση.

Για δύναμη χρησιμοποιείται ο τελεστής **:

```
x = 5 ** 2
```

8. Η ακέραια και η πραγματική διαίρεση

Αν οι δύο τελεστέοι είναι ακέραιοι, η διαίρεση είναι ακέραια.

```
integer :: a, b, c
```

```
a = 7  
b = 2  
c = a / b
```

Η τιμή της c είναι 3 και όχι 3.5. Αντίθετα, με πραγματικούς τελεστέους:

```
real :: a, b, c
```

```
a = 7.0  
b = 2.0  
c = a / b
```

η τιμή της c είναι 3.5. Όταν χρειάζεται μετατροπή ακέραιου σε πραγματικό, μπορεί να χρησιμοποιηθεί η συνάρτηση REAL:

```
real(a) / real(b)
```

Ο φοιτητής πρέπει να ελέγχει πάντα τους τύπους των μεταβλητών πριν αποφασίσει τι αποτέλεσμα περιμένει από μια διαίρεση.

9. Η συνάρτηση MOD

Η MOD(a,b) δίνει το υπόλοιπο της ακέραιας διαίρεσης. Για παράδειγμα:

`mod(17, 5)`

δίνει 2, επειδή $17 = 3 \times 5 + 2$.

Η MOD είναι ιδιαίτερα χρήσιμη όταν ο φοιτητής χρειάζεται να επεξεργαστεί ψηφία ενός ακέραιου αριθμού. Για παράδειγμα:

`583 / 10      -> 58      (ακέραια διαίρεση)`
`583 MOD 10   -> 3      (τελευταίο ψηφίο)`

Η τεχνική αυτή συνδέεται με ασκήσεις του αρχικού υλικού όπου ζητείται επεξεργασία των ψηφίων ενός αριθμού.

10. Είσοδος και έξοδος: READ και PRINT

Για να δώσει ο χρήστης δεδομένα στο πρόγραμμα χρησιμοποιείται η READ. Για να εμφανιστεί αποτέλεσμα χρησιμοποιείται η PRINT.

```
print *, 'Δώσε δύο ακεραίους:'  
read *, a, b
```

```
c = a + b
```

```
print *, 'Το άθροισμα είναι:', c
```

Η READ * διαβάζει τιμές από την τυπική είσοδο και η PRINT * εμφανίζει δεδομένα στην τυπική έξοδο. Η μορφοποίηση της εισόδου/εξόδου θα εξεταστεί αναλυτικότερα στο αντίστοιχο εργαστήριο.

11. Από μαθηματικό τύπο σε Fortran

Μια από τις σημαντικότερες δεξιότητες είναι η σωστή μετάφραση μαθηματικών εκφράσεων. Ο φοιτητής πρέπει πρώτα να γράφει καθαρά τον μαθηματικό τύπο και μετά να τον μετατρέπει στη σύνταξη της Fortran.

$x^2 + y^2 + z^2$
→ `x**2 + y**2 + z**2`

$(x + 1) / (y + 2)$
→ `(x + 1.0) / (y + 2.0)`

$(3x + 2) / (y^2 + 1)$
→ `(3.0*x + 2.0) / (y**2 + 1.0)`

Για κλάσματα, είναι καλό να χρησιμοποιούνται παρενθέσεις τόσο στον αριθμητή όσο και στον παρονομαστή. Έτσι μειώνεται ο κίνδυνος να αλλάξει κατά λάθος η μαθηματική σημασία της έκφρασης.

12. Πώς αναλύεται μια άσκηση

Ο φοιτητής δεν πρέπει να αντιμετωπίζει μια άσκηση ως «βρες τον σωστό κώδικα». Πρέπει να τη μετατρέπει σταδιακά από εκφώνηση σε αλγόριθμο και μετά σε Fortran.

Βήμα 1 — Εντοπίζει τα δεδομένα. Τι του δίνει η εκφώνηση;

Βήμα 2 — Εντοπίζει τα ζητούμενα. Τι πρέπει να υπολογίσει ή να εμφανίσει;

Βήμα 3 — Γράφει τους μαθηματικούς τύπους. Δεν γράφει ακόμη Fortran.

Βήμα 4 — Επιλέγει τύπους δεδομένων. Αν χρειάζεται δεκαδικό αποτέλεσμα, ελέγχει αν πρέπει να χρησιμοποιήσει REAL.

Βήμα 5 — Γράφει τις δηλώσεις.

Βήμα 6 — Γράφει READ/PRINT όπου χρειάζεται.

Βήμα 7 — Μετατρέπει τους τύπους σε εκφράσεις Fortran.

Βήμα 8 — Ελέγχει το πρόγραμμα με ένα απλό γνωστό παράδειγμα.

Αυτός ο τρόπος σκέψης θα χρησιμοποιείται σε όλα τα επόμενα κεφάλαια.

Παράδειγμα φυσικού προβλήματος: Για ευθύγραμμη ομαλά επιταχυνόμενη κίνηση, αν δίνονται s , u_0 και t , ο φοιτητής μπορεί πρώτα να ξεκινήσει από τη σχέση $s = u_0 t + \frac{1}{2} a t^2$ και να λύσει ως προς a . Μόνο μετά μετατρέπει τον τελικό τύπο σε Fortran.

13. Συχνά λάθη

1. Ξεχνά το `implicit none`. Ο φοιτητής πρέπει να συνηθίσει στη ρητή δήλωση τύπων.

2. Χρησιμοποιεί `INTEGER` όταν χρειάζεται `REAL`.

```
integer :: a, b
real :: x
a = 7
b = 2
x = a / b
```

Η έκφραση a/b είναι ακέραια διαίρεση. Αν χρειάζεται 3.5, πρέπει να γίνει κατάλληλη μετατροπή ή να χρησιμοποιηθούν πραγματικοί τελεστές.

3. Ξεχνά το `**` για δύναμη. Η Fortran χρησιμοποιεί `**` για ύψωση σε δύναμη.

4. Μεταφράζει λάθος ένα κλάσμα. Για $(x+1)/(y+2)$ πρέπει να διατηρούνται οι παρενθέσεις.

5. Γράφει κώδικα πριν καταλάβει την άσκηση. Ο φοιτητής πρέπει πρώτα να αναλύει δεδομένα, ζητούμενα και τύπους.

14. Ασκήσεις αυτοελέγχου

Οι παρακάτω ασκήσεις λειτουργούν ως προετοιμασία για τις πρώτες ασκήσεις του υλικού. Ο στόχος είναι να ελεγχθεί η κατανόηση, όχι να απομνημονευτεί ένας έτοιμος κώδικας.

Άσκηση 1 — Το πρώτο πρόγραμμα. Να γραφτεί πρόγραμμα που δηλώνει δύο ακεραίους, τους διαβάζει από τον χρήστη και εμφανίζει το άθροισμά τους.

Άσκηση 2 — Βασικές πράξεις. Να τροποποιηθεί το προηγούμενο πρόγραμμα ώστε να εμφανίζει άθροισμα, διαφορά και γινόμενο.

Άσκηση 3 — Διαίρεση. Να γραφτεί πρόγραμμα που διαβάζει δύο ακεραίους και εμφανίζει το ακέραιο πηλίκο και το υπόλοιπο. Να χρησιμοποιηθεί η MOD.

Άσκηση 4 — Πραγματικοί αριθμοί. Να γραφτεί πρόγραμμα που διαβάζει δύο πραγματικούς αριθμούς και υπολογίζει το πηλίκο τους. Να εξηγηθεί γιατί το αποτέλεσμα διαφέρει από την ακέραια διαίρεση.

Άσκηση 5 — Μετάφραση τύπου. Να μετατραπεί σε Fortran η έκφραση $(3x+2)/(y^2+1)$. Να δηλωθούν κατάλληλα οι μεταβλητές και να ελεγχθεί η έκφραση με συγκεκριμένες τιμές.

Άσκηση 6 — Ψηφία αριθμού. Για έναν τριψήφιο ακέραιο να βρεθεί το τελευταίο ψηφίο με MOD. Στη συνέχεια να διερευνηθεί πώς μπορεί να αφαιρεθεί το τελευταίο ψηφίο με ακέραια διαίρεση.

15. Checklist πριν προχωρήσει στο επόμενο κεφάλαιο

Ο φοιτητής είναι έτοιμος να προχωρήσει όταν μπορεί, χωρίς να αντιγράφει από τις σημειώσεις, να κάνει τα παρακάτω:

- Να γράψει τη βασική δομή ενός προγράμματος Fortran.
- Να χρησιμοποιήσει implicit none.
- Να δηλώσει INTEGER και REAL μεταβλητές.
- Να επιλέξει σωστό τύπο δεδομένων για ένα απλό πρόβλημα.
- Να χρησιμοποιήσει READ και PRINT.
- Να γράψει εκχώρηση της μορφής μεταβλητή = έκφραση.
- Να χρησιμοποιήσει +, -, *, / και **.
- Να εξηγήσει τη διαφορά ακέραιας και πραγματικής διαίρεσης.
- Να χρησιμοποιήσει MOD.
- Να μετατρέψει έναν απλό μαθηματικό τύπο σε Fortran.
- Να αναλύσει μια εκφώνηση σε δεδομένα, ζητούμενα και βήματα λύσης.

Κεντρική ιδέα του κεφαλαίου:

Δεν χρειάζεται να θυμάται απ' έξω έναν μεγάλο αριθμό εντολών. Χρειάζεται να μάθει να σκέφτεται με τη σειρά: τι γνωρίζει → τι ζητείται → ποιος τύπος χρειάζεται → τι τύπους δεδομένων χρησιμοποιεί → πώς γράφεται στη Fortran → πώς ελέγχει το αποτέλεσμα.

Πηγές του κεφαλαίου

FORTTRAN_2025.pdf — βασική δομή προγράμματος, ονόματα, implicit none, τύποι δεδομένων, τελεστές και εκχώρηση.

Περιγραφή Μαθήματος.docx — θέση των βασικών στοιχείων Fortran, εκφράσεων/προτάσεων και προτεραιότητας τελεστών μέσα στην ύλη.

Ασκήσεις_01.pdf και Εργαστήριο_2_Εκφράσεις και Προτάσεις_Μορφοποίηση.pdf — εφαρμογές και ασκήσεις πάνω στις πρώτες έννοιες.

Συμπλήρωμα στην §10 — Είσοδος και έξοδος: READ, PRINT και WRITE

Η ενότητα αυτή προστίθεται στο τέλος των αρχικών σημειώσεων ώστε να μη χαθεί ή αλλάξει κανένα από τα αρχικά κεφάλαια. Επεκτείνει μόνο την ενότητα της εισόδου/εξόδου με περισσότερη ανάλυση και παραδείγματα.

1. Η βασική ιδέα

Η READ χρησιμοποιείται όταν το πρόγραμμα πρέπει να πάρει δεδομένα από μια μονάδα εισόδου. Στα απλά προγράμματα του κεφαλαίου, αυτή είναι συνήθως το πληκτρολόγιο. Η PRINT και η WRITE χρησιμοποιούνται για έξοδο. Το υλικό του μαθήματος παρουσιάζει τις τρεις αυτές προτάσεις και διακρίνει την απλουστευμένη από την πλήρη μορφή τους. filecite δεν εμφανίζεται μέσα στο PDF.

2. Τι σημαίνει READ *

Η μορφή `read *`, `x` είναι η σύντομη μορφή που χρησιμοποιείται στα πρώτα προγράμματα για να διαβαστεί μια τιμή από την προκαθορισμένη είσοδο με ελεύθερη μορφοποίηση. Το ίδιο ισχύει για περισσότερες μεταβλητές:

```
read *, x
read *, a, b, c
```

Στο υλικό του μαθήματος εμφανίζεται ακριβώς αυτή η λογική: `read *, x` «περιμένει την τιμή από τον χρήστη» και `read *, a, b, c` διαβάσει τις τρεις τιμές από το πληκτρολόγιο. filecite δεν εμφανίζεται μέσα στο PDF.

3. Τι σημαίνει READ(*,*)

Η πλήρης σύνταξη για την ίδια απλή περίπτωση γράφεται `read(*,*) a, b, c`. Εδώ φαίνεται ρητά η λίστα ελέγχου της READ. Το πρώτο `*` αντιστοιχεί στην προκαθορισμένη μονάδα εισόδου και το δεύτερο `*` δηλώνει ελεύθερη μορφοποίηση (list-directed input). Το υλικό δίνει τη γενική σύνταξη ως `read (λίστα_ελέγχου)[λίστα_δεδομένων]` και τη σύντομη μορφή ως `read μορφή[, λίστα_δεδομένων]`. filecite δεν εμφανίζεται μέσα στο PDF.

4. Η ουσιαστική διαφορά ανάμεσα στις δύο μορφές

Μορφή	Τι γράφει ρητά	Χρήση στο επίπεδο του κεφαλαίου
<code>read *, a, b</code>	Χρησιμοποιεί τη σύντομη μορφή της READ.	Είναι ο πιο απλός τρόπος για τα πρώτα προγράμματα και τις απλές ασκήσεις.
<code>read(*,*) a, b</code>	Δείχνει ρητά τη μονάδα εισόδου και τη μορφή εισόδου.	Χρησιμοποιείται όταν θέλει να γράψει την πλήρη μορφή της πρότασης.

Πρακτικά: για τις απλές περιπτώσεις του κεφαλαίου, ο φοιτητής μπορεί να θεωρεί ότι οι δύο μορφές οδηγούν στην ίδια απλή αλληλεπίδραση με το πληκτρολόγιο. Η δεύτερη απλώς κάνει ρητά αυτά που η πρώτη αφήνει στη σύντομη μορφή. Δεν πρέπει όμως να μάθει ότι τα δύο αστεράκια «σημαίνουν το ίδιο πράγμα»: έχουν διαφορετικό ρόλο μέσα στη λίστα ελέγχου.

5. Παράδειγμα βήμα-βήμα με READ *

```
program example
implicit none
integer :: a, b

print *, 'give a and b'
read *, a, b
print *, 'sum = ', a + b

end program example
```

Αν δοθούν 4 7, η READ παίρνει τις δύο τιμές και τις αποθηκεύει στις `a` και `b`. Έπειτα η PRINT εμφανίζει το αποτέλεσμα της έκφρασης `a+b`, δηλαδή 11.

6. Το ίδιο παράδειγμα με READ(*,*) και WRITE(*,*)

```

program example
implicit none
integer :: a, b

write (*,*) 'give a and b'
read (*,*) a, b
write (*,*) 'sum = ', a + b

end program example

```

Εδώ η λογική του προγράμματος είναι ακριβώς η ίδια: πρώτα εμφανίζεται μήνυμα, μετά διαβάζονται δύο τιμές και στο τέλος εμφανίζεται το άθροισμα. Η αλλαγή είναι ότι γράφτηκε η πλήρης μορφή και για την είσοδο και για την έξοδο.

7. PRINT και WRITE

Η PRINT είναι η απλή μορφή εξόδου που θα χρησιμοποιείται συχνά στα πρώτα προγράμματα:

```

print *, x
print *, 'x = ', x

```

Η WRITE είναι η γενικότερη πρόταση εξόδου. Στην απλή περίπτωση ελεύθερης μορφοποίησης γράφεται:

```

write (*,*) x
write (*,*) 'x = ', x

```

Το υλικό του μαθήματος δίνει ρητά το ζεύγος `read(*,*)` / `write(*,*)` ως απλές προτάσεις εισόδου-εξόδου με πλήρη μορφή. filecite δεν εμφανίζεται μέσα στο PDF.

8. Γιατί να μάθει και τις δύο μορφές;

Στα πρώτα απλά προγράμματα, η `read *` είναι πολύ βολική επειδή είναι σύντομη και ευανάγνωστη. Η `read(*,*)` είναι χρήσιμη επειδή τον εξοικειώνει με τη γενική σύνταξη της Fortran και θα τον βοηθήσει όταν συναντήσει αργότερα πιο σύνθετες λίστες ελέγχου, αρχεία ή ρητή μορφοποίηση. Στο σημείο αυτό δεν χρειάζεται να μάθει τις προχωρημένες επιλογές της READ· χρειάζεται να καταλάβει τη διαφορά ανάμεσα στη σύντομη και την πλήρη μορφή.

9. Τι πρέπει να θυμάται

- `read *, x` σύντομη μορφή, διαβάζει από την προκαθορισμένη είσοδο με ελεύθερη μορφοποίηση.
- `read(*,*) x` πλήρης μορφή για την ίδια απλή περίπτωση.
- Το πρώτο `*` στο `read(*,*)` αφορά τη μονάδα εισόδου.
- Το δεύτερο `*` αφορά την ελεύθερη μορφοποίηση.
- `print *, x` απλή έξοδος.
- `write(*,*) x` έξοδος με την πλήρη μορφή της απλής ελεύθερης μορφοποίησης.

10. Μικρό παράδειγμα για αυτοέλεγχο

```

program test
implicit none
real :: x, y

print *, 'give x and y'
read (*,*) x, y
print *, 'product = ', x * y

end program test

```

Αν ο χρήστης δώσει 2.0 4.0, ο φοιτητής πρέπει να μπορεί να εξηγήσει: η READ αποθηκεύει 2.0 στο `x` και 4.0 στο `y`, η έκφραση `x*y` δίνει 8.0 και η PRINT εμφανίζει το αποτέλεσμα. Η ακριβής εμφάνιση των δεκαδικών ψηφίων εξαρτάται από την προεπιλεγμένη μορφοποίηση.